

Algebraic Coincidence Classes in ECDSA: Chance-Rate Occurrence and a Constructive Separation

*A negative result on three signature classes, with one actionable consequence
for anyone tempted to scan for them*

Andriy Polishchuk

CRYPTON Systems Lab (independent research, no institutional affiliation)

andriy.polishchuk.a@gmail.com

Artifact: github.com/apolish/ECDSA-Research

Version 2 · August 2026

Abstract

An ECDSA signature satisfies the additive split $s = s_{zk} + s_{rxk} \pmod n$, where $s_{zk} = z \cdot k^{-1}$ is secret. This paper studies three algebraic conditions - here called classes A, B and E - under which s_{zk} is pinned down by a formula in the public data (z, r, s) alone, so that the private key follows deterministically and is confirmed on the curve. We report a negative result and one positive by-product.

Negative result. Class membership is not decidable in advance from public data: each class is the a posteriori name for the event 'a fixed guess happened to be correct'. We derive closed forms for the occurrence probabilities - $P(A) = 1/(4(n-1))$ and $P(E) = 4/(3n)$ - and confirm them against a structure-free null model over 10^8 signatures on a small curve. Genuine ECDSA signatures and null tuples are statistically indistinguishable in every class (all $|z| \leq 1.4$). On secp256k1 the rates are 2^{-258} and $2^{-255.6}$, i.e. no better than guessing the key outright. Class B is rarer still and, we argue constructively, unreachable there by any means short of brute force.

Positive by-product. The same algebra yields a clean separation. Conditions that constrain only (s_{zk}, s_{rxk}) - class E - are freely constructible by a signer on any curve; class A needs only that $P = k \cdot r \pmod n$ be odd, about two nonces on average; class B pins the ratio $s_{zr} : s_{zk}$ to a fixed rational and is therefore bound to the random-oracle behaviour of $k \rightarrow k \cdot (kG)_x$. We construct class-A and class-E signatures on real secp256k1 in about 40 ms each. The practical consequence is a warning: because these signatures are cheap to manufacture, class membership of an on-chain signature is never evidence that the signer's key is weak, and any scanner reporting it as such can be poisoned at roughly $2 \cdot 10^6$ signatures per core-day.

Also. We correct a factor-of-two undercount in the previously published class-E figure for this artifact, caused by a detector that silently discarded half of the class.

1. Introduction and scope

The Elliptic Curve Digital Signature Algorithm is unforgiving about its nonce. If the same k is used twice the key falls out of two signatures by elementary algebra, as the PlayStation 3 signing key famously did [1]. If k is merely biased, lattice methods recover the key from a few dozen to a few thousand signatures [2, 3, 4], and this has been carried out against real deployments and real cryptocurrency wallets [5, 6, 7, 8]. These results share two properties that make them useful: the precondition is either publicly detectable (a repeated r) or statistically accumulable over many signatures (a bias).

This paper examines a family of conditions that has neither property, and says so plainly. Working from the additive decomposition of s , one can write down formulas in the public data (z, r, s) that sometimes coincide with the secret quantity $s_zk = z \cdot k^{-1}$. When such a coincidence occurs, the private key follows deterministically and can be confirmed on the curve using public data only. It is tempting to call these 'classes of recoverable signatures'. That framing is wrong, and the bulk of this paper is an argument - part logical, part experimental - for why it is wrong.

We state the scope bluntly, because work in this area attracts overstatement:

What this paper does and does not claim

It is not an attack. Nothing here recovers a key from a signature the author did not create. No weakness in ECDSA, in secp256k1, in RFC 6979 [9], or in any implementation is claimed, and none was found.

The classes are not detectable in advance. Membership is determined by testing a guess against the curve. The formulas add a name, not information.

The measured rates match chance. On a small curve where the events are observable, genuine signatures and structure-free random tuples produce them at statistically indistinguishable rates.

The one operational claim is defensive. Class-A and class-E signatures can be manufactured cheaply on real secp256k1, so their presence must never be read as evidence of a weak key.

The contribution is therefore threefold: an explicit account of why an appealing-looking structure carries no information (Section 3); closed-form and measured occurrence rates validated against a null model, together with a correction to a previously published figure (Section 4); and a constructive separation of the three classes by what an adversarial signer can and cannot steer, with measured costs on real secp256k1 (Sections 5-6). Section 7 records the threats to validity, including the one that most threatens this kind of work: the classes were found by exploration, and exploration always finds something.

All code, tests and data are in the artifact repository, which runs on a stock Python 3.8+ installation with no third-party dependencies.

2. Notation and the additive split

Let E be an elliptic curve over F_p with base point G of prime order n . For a private key x , message hash z and nonce k , an ECDSA signature [10] is

$$\begin{aligned} r &= (k \cdot G)_x \bmod n \\ s &= (z + r \cdot x) \cdot k^{-1} \bmod n \end{aligned}$$

The whole paper rests on splitting s additively. Define

$$\begin{aligned}s_{zk} &= z \cdot k^{-1} \bmod n \\ s_{rxk} &= r \cdot x \cdot k^{-1} \bmod n \\ s_{zr} &= z \cdot r \bmod n\end{aligned}$$

so that $s = s_{zk} + s_{rxk} \pmod n$ identically. Of these, s_{zr} is public (it is a product of two published values) while s_{zk} and s_{rxk} are secret: each depends on k^{-1} .

The single primitive used throughout is that a correct value of s_{zk} reveals the key. From the signature identity,

$$x = z \cdot (s - s_{zk}) \cdot (r \cdot s_{zk})^{-1} \bmod n$$

Every input is public. A candidate produced this way is then confirmed, again using public data only, by recomputing $k = (z + r \cdot x) \cdot s^{-1}$ and testing $(k \cdot G)_x \bmod n = r$. This check is decisive: it accepts the true key and rejects everything else, so no unverified candidate is ever reported as a result.

The problem, of course, is that s_{zk} is not public. The classes below are attempts to guess it from a formula.

3. The three classes, and why membership carries no information

3.1 Definitions

All classes are defined through the quantity

$$a = s \bmod ((s_{zr} - s) \bmod n), \text{ defined when } s_{zr} > s \text{ and } 0 < a \neq s$$

which is computable from public data. Write $m1 = s_{zk}/a$ and $m2 = (s + s_{zr})/s_{rxk}$ as exact rationals. The taxonomy is then:

Class	Condition	Is s_{zk} pinned by public data?
A	$m1 = 1$, i.e. $s_{zk} = a$	Yes - directly, $s_{zk} = a$
B	$m1 = m2$, both integral	Yes - as a root of $a \cdot m1^2 - s \cdot m1 + (s + s_{zr}) = 0$
E	$s_{zk} + s_{rxk} = S$ and $ s_{zk} - s_{rxk} = S//2 + 1$, for $S \in \{s, s+n\}$ with $S \bmod 4 \in \{1, 2\}$	Yes - both parts determined by S alone
C	$m1$ integral, $m1 > 1$	No - $m1$ is not a function of public data
D	$m1$ not integral	No

Table 1. The taxonomy. Only A, B and E yield a key; C and D are recorded for completeness because together they account for essentially all signatures.

Class E deserves a word on naming. Its detector tests whether the integer sum S of the two parts splits so that their difference is exactly $S//2 + 1$; the split $u = (S + S//2 + 1)//2$, $v = S - u$ is integral precisely when $S \equiv 1$ or $2 \pmod{4}$. Since $s \equiv s_{zk} + s_{rxk} \pmod n$, the true integer sum is exactly one of s (label E1) or $s + n$ (label E2), so both must be tried. Note that this construction treats elements of $\mathbb{Z}_n$ as integers - a point we return to in Section 7.

3.2 The epistemic problem

Suppose a signature is in class A. Then $s_{zk} = a$, the key follows from the recovery formula, and the curve confirms it. Deterministic, exact, no search. This sounds like a result.

It is not, because *membership cannot be decided before the fact*. Given a signature, an observer computes a , assumes $s_{zk} = a$, derives a candidate key and tests it on the curve. If the test passes, the signature is declared to be in class A. Class A is thus not a precondition that enables recovery; it is the a posteriori name for the event *my guess happened to be right*.

Stated without the algebra, the procedure is: guess a value for the secret; check it; if it was correct, announce that this signature belongs to the class of signatures whose secret equals that guess. The formula for a supplies a specific guess rather than a random one, but supplying a guess is not the same as supplying information. The relevant question is whether the guess is right more often than a uniformly random one would be - and Section 4 answers no.

The contrast with the genuine nonce results is instructive:

Precondition	Detectable in advance?	Accumulates over signatures?	Cost once present
Repeated nonce [1]	Yes - equal r values are public	n/a - two suffice	Trivial algebra
Biased nonce [2, 3, 4]	No, but statistically inferable	Yes - that is the mechanism	Lattice reduction
Classes A / B / E (this work)	No	No	Trivial algebra

Table 2. Why the classes are not in the same category as the known nonce failures. Lacking both a detectable precondition and any accumulation across signatures, each signature is an independent lottery ticket.

Because the events do not accumulate, collecting more signatures does not help: N signatures give N independent chances at a probability the next section shows to be about $1/n$. That is precisely the arithmetic of guessing keys at random.

4. Measured occurrence against a null model

4.1 Closed forms

Model s_{zk} and s_{rxk} as independent uniform values on $[1, n)$ - which they are, for uniform z and x - and s_{zr} as uniform and independent. Two probabilities follow.

The gate. A well-defined a requires $s < s_{zr} \leq 2s$: the left inequality is the definition, and the right excludes $a = s$. For independent uniforms this integrates to exactly $1/4$.

Class A. Conditioned on the gate, a is a function of (s, s_{zr}) while s_{zk} remains uniform on $[1, n)$ given s . Hence

$$P(A) = (1/4) \cdot 1/(n-1)$$

Class E. Enumerate over the integer sum $S = s_{zk} + s_{rxk} \in [2, 2n)$. The split is integral for the half of all S with $S \bmod 4 \in \{1, 2\}$, and then the two parts are determined; both lie in $[1, n)$ only while $S < 4n/3$, and each admissible S contributes two ordered pairs. So the favourable count is $2 \cdot (1/2) \cdot (4n/3) = 4n/3$ out of $(n-1)^2$, giving

$$P(E) = 4/(3n)$$

Exhaustive enumeration on the test curve gives 132,884 favourable ordered pairs, i.e. $1.337761 \cdot 10^{-5}$ against the closed form $1.337788 \cdot 10^{-5}$ - agreement to four decimal places.

4.2 Experimental design

A closed form derived under an independence assumption proves nothing about real signatures, since the assumption is exactly what is in question. We therefore compare two streams classified by identical code:

Real. Honest ECDSA signatures on the test curve secp17k1 ($p = 100003$, $b = 2$, $n = 99667$ prime, cofactor 1), with z , k and x drawn uniformly and r read from an exact table of $(k \cdot G)_x \bmod n$.

Null. s_{zk} and s_{rxk} drawn uniformly and independently; $s = s_{zk} + s_{rxk} \bmod n$; and $s_{zr} = s_{zk} \cdot P$ with P uniform. This keeps the additive split - which is definitional and must hold - but replaces the curve-determined product $P = k \cdot r$ by an independent uniform value. The null model is the hypothesis under test.

Each stream comprises 10^8 tuples (seed 20260812). The vectorised classifier was checked against the repository's own detector on 200,000 tuples with zero disagreements.

4.3 Results

Quantity	Real (10^8)	Rate	Null (10^8)	Rate	Closed form	z
gate (valid a)	24,990,575	0.249906	24,992,228	0.249922	0.250000	-0.23
case A	220	$2.20 \cdot 10^{-6}$	250	$2.50 \cdot 10^{-6}$	$2.5084 \cdot 10^{-6}$	-1.38
case B	0	$< 10^{-8}$	0	$< 10^{-8}$	$\approx 5 \cdot 10^{-10}$	0.00
case C	55,548	$5.5548 \cdot 10^{-4}$	55,664	$5.5664 \cdot 10^{-4}$	-	-0.35
case E	1,355	$1.3550 \cdot 10^{-5}$	1,344	$1.3440 \cdot 10^{-5}$	$1.3378 \cdot 10^{-5}$	+0.21

Table 3. Class census on 10^8 genuine signatures and 10^8 null tuples. The final column is the two-sample Poisson statistic $z = (\text{real} - \text{null})/\sqrt{\text{real} + \text{null}}$. No class shows a deviation beyond 1.4 standard deviations.

Every measured rate agrees with its closed form, and - more to the point - genuine ECDSA signatures are statistically indistinguishable from structure-free tuples in every class. The largest discrepancy, class A at $z = -1.38$, corresponds to $p \approx 0.17$ under the null: entirely unremarkable, and in the direction of *fewer* events than chance predicts.

Class B is too rare to measure at this sample size. A dedicated run of $2 \cdot 10^9$ null tuples produced a single event, a rate near $5 \cdot 10^{-10}$ with a Poisson 95% interval of roughly $[1.3 \cdot 10^{-10}, 2.8 \cdot 10^{-9}]$ - three to four orders of magnitude below classes A and E even on a toy curve. Its status on secp256k1 is settled in Section 5 by a constructive argument rather than by frequency estimation.

4.4 A correction to the previously published figure

The artifact repository previously reported 670 class-E events in a 10^8 -signature run on this curve, a rate of $6.70 \cdot 10^{-6}$. That figure is an undercount by a factor of about two. Splitting our own class-E count by residue gives 659 events with $S \equiv 1 \pmod{4}$ and 658 with $S \equiv 2 \pmod{4}$. The earlier detector required S to be even and so silently discarded the entire $S \equiv 1 \pmod{4}$ family; 658 is within Poisson noise of the published 670. The corrected rate is $1.34 \cdot 10^{-5}$, matching $4/(3n)$.

We record this not because it changes any conclusion - it does not; twice a negligible rate is still negligible - but because it is the kind of error that inflates a claim in the favourable direction, and it was found only by deriving the closed form and noticing the measurement disagreed with it.

4.5 Projection to secp256k1

The closed forms are functions of n alone, so they transfer directly:

Class	Probability	secp17k1 ($n = 99667$)	secp256k1 ($n \approx 1.16 \cdot 10^{77}$)
A	$1/(4(n-1))$	$2.51 \cdot 10^{-6}$	$2.16 \cdot 10^{-78} \approx 2^{-258}$
E	$4/(3n)$	$1.34 \cdot 10^{-5}$	$1.15 \cdot 10^{-77} \approx 2^{-255.6}$
B	empirically $\approx 10^{-9}$ here; see §5	$\approx 5 \cdot 10^{-10}$	unreachable by construction

Table 4. Occurrence probabilities per signature. For comparison, guessing the private key outright succeeds with probability $1/n = 2^{-256}$.

The comparison in the caption is the entire point. Waiting for a class-A signature on secp256k1 is *worse* than guessing the key at random, and class E is better only by a factor of about 1.3. No amount of engineering changes this, because it is a statement about the size of n , not about any implementation.

5. A constructive separation of the three classes

The negative result above says the classes are not useful to an observer. A different question - what a *signer* can arrange deliberately - has a sharper answer, and it is the one genuinely new piece of structure here.

5.1 The construction

Choose a nonce k (hence r) and the two additive parts (s_{zk} , s_{rxk}) freely. Everything else is then forced:

$$\begin{aligned} z &= s_{zk} \cdot k \bmod n \\ x &= s_{rxk} \cdot k \cdot r^{-1} \bmod n \\ s &= s_{zk} + s_{rxk} \bmod n \end{aligned}$$

The result is an ordinary ECDSA signature: it verifies against $Q = x \cdot G$ by the standard equation. So a signer who wants a signature with prescribed (s_{zk} , s_{rxk}) simply picks them.

5.2 The one quantity that cannot be steered

The construction gives away z and x , but not everything. Since $s_{zr} = z \cdot r = s_{zk} \cdot (k \cdot r)$, the product

$$P = k \cdot r = k \cdot (k \cdot G)_x \bmod n$$

is fixed the moment k is chosen, and the map $k \rightarrow k \cdot (k \cdot G)_x$ behaves as a random oracle: no method is known to steer it to a chosen value short of trying nonces. Every class condition can therefore be sorted by whether it constrains P .

Class	Constrains P?	Cost per nonce	Test curve	Real secp256k1
E	No - the condition involves only s_{zk} and s_{rxk}	none; any nonce works	yes	yes
A	Weakly - with $s_{zk} = 1$ it needs only P odd	$1/2$, i.e. about 2 nonces	yes	yes
B	Yes - pins the ratio $s_{zr} : s_{zk}$ to a fixed rational	$1/n \approx 2^{-256}$	yes, by enumerating all nonces	no

Table 5. Constructibility by class. The separation is caused entirely by the dependence on the uncontrollable product $P = k \cdot r \bmod n$.

Class E is the extreme case: its condition mentions neither r nor s_{zr} , so any nonce whatsoever will do. Class A is nearly as easy: taking $s_{zk} = 1$ reduces the requirement to the existence of a divisor $D > 1$ of $s_{zr} - 1$, and $D = 2$ serves whenever P is odd - so about two nonces suffice, with no factoring.

Class B is different in kind. Requiring $m_1 = m_2$ with both integral fixes the ratio $s_{zr} : s_{zk}$ to one of a discrete set of rationals, which is to say it fixes P to a specific value. On the test curve all n nonces can be enumerated and indexed by P , so a hit is found immediately; we exhibit such signatures. On secp256k1 the same requirement is a 2^{-256} search per nonce. A probe of 3,000 nonces produced no hit, as expected - the expectation is $2.6 \cdot 10^{-74}$. Our implementation therefore refuses class B in legacy mode with an explanatory error rather than looping indefinitely.

This yields the paper's one clean positive statement: **classes A and E are signer-forgeable on real secp256k1; class B is not**. The distinction is not a matter of degree but of whether the defining condition touches P .

5.3 Measured cost

We constructed 100 signatures of each forgeable class on real secp256k1, verifying each one twice: the private key is re-derived from the public (z, r, s) by the recovery routine, and the candidate is confirmed on the curve.

Class	Signatures	Total time	Per signature	Throughput	Per core-day
A	100	3.75 s	37.5 ms	26.7 / s	$2.3 \cdot 10^6$
E	100	4.34 s	43.4 ms	23.0 / s	$2.0 \cdot 10^6$

Table 6. Cost of constructing class-A and class-E signatures on real secp256k1, single core, pure Python with no optimisation. A compiled implementation would be orders of magnitude faster.

For calibration: producing one class-A signature on secp256k1 by waiting for it to occur naturally would take about $4.6 \cdot 10^{77}$ signatures. Constructing one takes 37.5 milliseconds. That gap is the subject of the next section.

5.4 Why this is not an attack

The construction requires choosing z as $s_{zk} \cdot k$, so z is not the hash of any particular message; the artifact marks every constructed signature as not message-bound and says so in the report preamble. More fundamentally, the construction consumes x and k as inputs. It is a statement about what a signer can do to their own signatures, and it transfers to nobody else's: a real signer's x is secret and its z is a fixed message hash.

Binding the construction to a chosen message would additionally require inverting the hash - which is the usual barrier, unmoved by anything here.

6. The actionable consequence: membership is not evidence

Sections 4 and 5 combine into a warning worth stating on its own, because it is the only part of this work with operational content.

Class membership must never be read as evidence of a weak key

There is a natural temptation, given a classifier like this one, to run it across a public ledger and flag the signatures it matches - reasoning that a signature from which the key can be recovered indicates a broken signer. Precedent exists for exactly this kind of survey: large-scale scans have found genuinely weak keys in deployed systems [11, 12], and lattice-based surveys have recovered real cryptocurrency keys from biased nonces [7].

For classes A, B and E the inference does not hold, for two independent reasons. First, the base rate: on secp256k1 the classes occur at about 2^{-256} , so any real match is overwhelmingly more likely to be a deliberate construction than a spontaneous coincidence. Second, and decisively, constructions are cheap - roughly $2 \cdot 10^6$ signatures per core-day by Table 6, from a pure-Python implementation. A party who wishes to pollute such a scan can do so at negligible cost, and the resulting signatures are perfectly valid.

A scanner built on these classes therefore reports, in effect, 'this signer chose to produce a signature in this class'. That is a statement about intent, not about key strength - and it is trivially spoofable in either direction. The correct operational conclusion is to not build the scanner.

This asymmetry - an event that essentially never happens by chance but is cheap to manufacture - is worth naming in general. A condition is a useful weakness indicator only if it is expensive for the observed party to produce deliberately. Classes A and E fail that test on real secp256k1. Class B passes it, and is precisely the class that never occurs at all.

7. Threats to validity

This section is longer than is customary because the failure modes of this kind of work are well known and mostly self-inflicted.

7.1 The garden of forking paths

The formulas defining a , m_1 and m_2 were not derived from a structural property of ECDSA. They were found by exploring candidate expressions in (z, r, s) and keeping those that occasionally coincided with s_{zk} . Any such search, run over enough candidates, is guaranteed to produce expressions that sometimes coincide - the multiple-comparisons problem in its exploratory form [13].

We disclose this because it is the single most important caveat on the taxonomy, and because a reader who assumes a principled derivation would reasonably ask where it is. There is none. The null-model comparison in Section 4 is the appropriate response: rather than defending the formulas, we measure whether they beat chance, and they do not. A reader should treat the classes as an existence proof that such expressions can be written down, not as evidence of hidden structure.

7.2 The test curve is not a signing oracle

In test mode the artifact draws z uniformly rather than hashing a message. This is deliberate - it makes the uniformity assumption exact and lets the algebra be exercised at feasible scale - but it means test-curve statistics describe random triples, not signatures a verifier would accept against a message. Every rate in Section 4 should be read with that qualification. Conversely, the legacy path is fully message-bound: z is the double-SHA-256 of a Bitcoin sighash preimage and k is derived per RFC 6979 [9], but at that scale the classes are unobservable by construction.

7.3 Scale extrapolation

Everything measured here was measured on a 17-bit curve. The closed forms depend on n only, and the null model assumes nothing curve-specific, so the extrapolation in Table 4 is a statement about the model rather than a measurement. The claim being extrapolated is a null result, which is the benign direction: we are asserting that nothing happens at large n , having verified that nothing happens beyond chance at small n .

7.4 Representative dependence

The classification is not intrinsic to the group. Tests such as $s_{zr} > s$, $m1 > 1$ and the parity conditions in class E treat elements of Z_n as integers in $[0, n)$. Since n is odd, s and $s + n$ are the same group element with different parity, and class E exploits exactly this by trying both. Any statistic here therefore describes the chosen integer representatives, not the group - a further reason to regard the taxonomy as descriptive.

7.5 What would change the conclusion

For completeness, the result would have to be revisited if any of the following were shown: that $k \rightarrow k \cdot (k \cdot G)_x \bmod n$ admits a preimage method materially better than brute force, which would reopen class B and make class A steerable at will; that a real-world signer produces s_{zk} non-uniformly in a way correlated with these formulas, which would break the null model's premise; or that some class condition can be decided from public data before recovery is attempted, which would overturn Section 3 entirely. We have no evidence for any of these.

8. Related work

The literature on ECDSA nonce failures is mature and instructive as a contrast. Repeated nonces break the scheme outright and have done so in practice [1]. Partial nonce leakage is handled by reduction to the hidden number problem and solved with lattice techniques [2, 3, 4]; refinements and side-channel realisations continue to lower the required leakage, down to well under one bit [4, 5, 6]. Large-scale studies have applied these methods to deployed systems and to cryptocurrency ledgers, recovering real keys [7, 8]. Deterministic nonce generation [9] removes the entropy dependence that most of these attacks exploit.

What distinguishes those results from the present work is not the algebra but the precondition. In each case the precondition is either publicly visible or statistically accumable, and the number of signatures required is small and polynomially bounded. Here the precondition is invisible before the fact, does not accumulate, and occurs at the rate of a random guess. The methodological literature on exploratory analysis [13] is, in our view, the more relevant reference for interpreting findings of this shape.

We are not aware of prior work naming these particular coincidence classes. Given the analysis above, we would regard their absence from the literature as unsurprising rather

than as an opportunity.

9. Conclusion

Three algebraic conditions on ECDSA signatures were examined under which the secret quantity $s_{zk} = z \cdot k^{-1}$ is pinned down by public data, so that the private key follows deterministically. The determinism is real. Its value is nil, for a reason that is structural rather than incidental: membership is not decidable before recovery is attempted, so the classes name the outcome of a guess rather than enable one. Measured against a structure-free null model over 10^8 signatures, genuine ECDSA produces these events at chance rates in every class, with no deviation beyond 1.4 standard deviations. On secp256k1 the rates are near 2^{-256} - the probability of guessing the key outright.

The by-product is more durable than the main line of inquiry. Sorting the classes by whether they constrain the uncontrollable product $P = k \cdot r \bmod n$ separates them cleanly: classes A and E are constructible by a signer on real secp256k1 in tens of milliseconds, while class B is bound to that product and is therefore out of reach. The defensive consequence follows immediately and is the one recommendation this paper makes: because these signatures are cheap to manufacture and essentially never arise by chance, their presence on a ledger indicates a choice by the signer and never a weakness in the signer's key.

The wider lesson concerns method rather than cryptography. An expression found by exploration will sometimes coincide with a secret; the question is never whether it coincides but whether it coincides more often than chance. That question has a cheap and decisive answer - construct a null model that destroys the structure under test and measure both - and we would encourage anyone reporting a pattern in signature data to run it before publishing rather than after.

10. Artifact and reproducibility

All code, tests and generated data are available at github.com/apolish/ECDSA-Research. The implementation is pure Python 3.8+ with no third-party dependencies; the experiments in Section 4 additionally use NumPy for vectorisation only, and the same classification is available in scalar form.

The repository contains curve arithmetic with RFC 6979 nonce generation validated against the published test vectors, Bitcoin legacy and BIP 143 sighash preimage construction, the classifier and report writer, the class-A/B/E constructor of Section 5, and a regression suite of 21 checks that reports the value each test actually measured. Runs are reproducible end to end from a seed, covering keys as well as nonces and hashes.

The class construction of Section 5 is reachable from the command line:

```
# construct class A, B and E signatures on the test curve
python3 src/utls/generate_transactions.py --tx-with-classes A B E

# on real secp256k1: A and E are constructed, B is refused with a reason
python3 src/utls/generate_transactions.py --tx-with-classes A E B \
--curve-mode legacy
```

Reported figures were produced with seed 20260812 on Python 3.12.3.

Acknowledgement of limitations

This is independent work carried out without institutional review, funding or external replication. The negative result is the claim the author is most confident in, precisely because it required arguing against the author's own initial hypothesis; readers should weight the constructive material in Section 5 accordingly and are invited to replicate from the artifact.

References

- [1] fail0verflow. Console Hacking 2010: PS3 Epic Fail. 27th Chaos Communication Congress (27C3), Berlin, 2010.
- [2] N. Howgrave-Graham and N. P. Smart. Lattice Attacks on Digital Signature Schemes. *Designs, Codes and Cryptography*, 23(3):283-290, 2001.
- [3] P. Q. Nguyen and I. E. Shparlinski. The Insecurity of the Digital Signature Algorithm with Partially Known Nonces. *Journal of Cryptology*, 15(3):151-176, 2002.
- [4] P. Q. Nguyen and I. E. Shparlinski. The Insecurity of the Elliptic Curve Digital Signature Algorithm with Partially Known Nonces. *Designs, Codes and Cryptography*, 30(2):201-217, 2003.
- [5] J. Jancar, V. Sedlacek, P. Svenda and M. Sys. Minerva: The Curse of ECDSA Nonces. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(4), 2020.
- [6] D. F. Aranha, F. R. Novaes, A. Takahashi, M. Tibouchi and Y. Yarom. LadderLeak: Breaking ECDSA with Less than One Bit of Nonce Leakage. *ACM CCS*, 2020.
- [7] J. Breitner and N. Heninger. Biased Nonce Sense: Lattice Attacks against Weak ECDSA Signatures in Cryptocurrencies. *Financial Cryptography and Data Security (FC)*, 2019.
- [8] D. Moghimi, B. Sunar, T. Eisenbarth and N. Heninger. TPM-FAIL: TPM meets Timing and Lattice Attacks. *USENIX Security Symposium*, 2020.
- [9] T. Pornin. Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA). *RFC 6979*, IETF, 2013.
- [10] D. Johnson, A. Menezes and S. Vanstone. The Elliptic Curve Digital Signature Algorithm (ECDSA). *International Journal of Information Security*, 1(1):36-63, 2001.
- [11] N. Heninger, Z. Durumeric, E. Wustrow and J. A. Halderman. Mining Your Ps and Qs: Detection of Widespread Weak Keys in Network Devices. *USENIX Security Symposium*, 2012.
- [12] J. W. Bos, J. A. Halderman, N. Heninger, J. Moore, M. Naehrig and E. Wustrow. Elliptic Curve Cryptography in Practice. *Financial Cryptography and Data Security (FC)*, 2014.
- [13] A. Gelman and E. Loken. The Garden of Forking Paths: Why Multiple Comparisons Can Be a Problem, Even When There Is No 'Fishing Expedition'. Department of Statistics, Columbia University, 2013.